

Streaming Latency Benchmarks: REDIS Streams vs Apache KAFKA for Real-Time Sports Data Feeds

Journal Title
XX(X):1-6
©The Author(s) 2026
Reprints and permission:
sagepub.co.uk/journalsPermissions.nav
DOI: 10.1177/ToBeAssigned
www.sagepub.com/

SAGE

Gustavo Pedro Ricou¹

Abstract

This study presents a comprehensive, reproducible benchmark suite that empirically evaluates how streaming architecture choices impact the timeliness and reliability of live football analytics. Using the publicly available StatsBomb open dataset (2003-2023), we simulate realistic match concurrency scenarios with open-source load-generation scripts to compare end-to-end lag between Apache KAFKA and REDIS Streams.

Unlike traditional comparisons, we frame the investigation around sports-specific decision-making requirements: How quickly can coaching staff, broadcasters, and betting platforms act on live event data? Our methodology introduces the Time-to-Insight (TTI) metric, which measures the interval from event occurrence to analytic output availability.

Using 40,660 real football events across 11 matches, and after correcting two instrumentation defects in earlier measurements (a process-relative cross-process clock and a saturating load generator), our corrected multi-broker matrix shows Apache KAFKA achieving significantly lower median TTI than REDIS Streams in both single-broker and clustered deployments (Holm–Bonferroni-corrected Mann–Whitney U, Cohen’s $d = -1.18$ overall, large effect). Single-broker deployments outperform clustered ones for both systems, and stronger durability settings (KAFKA `acks=all`; REDIS `appendfsync always`) measurably increase latency. These findings reverse the conclusion of the initial, contaminated analysis and underscore the importance of measurement validity in cross-process streaming benchmarks.

Keywords

Streaming systems, real-time analytics, sports data, Apache Kafka, Redis Streams, latency benchmarking, Time-to-Insight, actionability windows, reproducible research, football analytics, research questions, hypotheses, statistical analysis

Introduction

The rapid digitization of sports has created an insatiable demand for real-time analytics. Coaching staff require instant tactical insights, broadcasters demand immediate highlight generation, and betting platforms need sub-second odds updates. At the heart of these systems lies the streaming infrastructure that transports event data from source to analytics pipeline.

Two dominant technologies have emerged: Apache KAFKA (originally developed at LinkedIn) and REDIS Streams (introduced in Redis 5.0). KAFKA is a distributed event streaming platform designed for high-throughput, fault-tolerant data pipelines. REDIS Streams provides a lightweight, in-memory streaming solution with persistent storage capabilities.

While numerous studies have compared these technologies, the literature lacks sports-specific evaluations. Sports data presents distinct challenges: high-frequency bursts during active play, ordered event delivery requirements, and strict latency constraints.

Sports-Specific Latency Requirements

Real-time sports analytics imposes domain-specific latency constraints that vary by use case and stakeholder. As summarized in Table 1, different applications demand

fundamentally different performance characteristics. Betting platforms require sub-500ms latency for in-play odds updates, as even a 2-second delay can erode user trust and cost millions in missed wagers Solutions (2025); Ververica (2025). Broadcasting applications can tolerate up to 3 seconds for live video synchronization OptiView (2025); Promwad (2025), while coaching decision systems need <500ms to enable real-time tactical adjustments Pappas et al. (2020); Sports (2023).

The concept of an “actionability window” formalizes this requirement: the maximum latency within which insights can still influence decisions. Using an exponential decay model ($Value = V_0 \times e^{-\lambda \times latency}$), we estimate that at 500ms latency, only ~60% of coaching insight value remains, dropping to ~13% at 2000ms. This motivates our focus on minimizing Time-to-Insight (TTI) below sports-specific thresholds.

¹School of Computer Science and Statistics, Trinity College Dublin, Dublin, Ireland

Corresponding author:

Gustavo Pedro Ricou, School of Computer Science and Statistics, Trinity College Dublin, Dublin, Ireland.

Email: pedrorigo@tcd.ie

Table 1. Sports Analytics Latency Requirements by Use Case and Stakeholder

Use Case	Stakeholder	Latency Range
Betting Platforms		
Live Odds Update	Betting Platform	100–500ms
In-Play Betting	Betting Platform	50–200ms
Broadcasting		
Live Video Sync	Broadcaster	500–3000ms
Live Stats Overlay	Broadcaster	100–1000ms
Interactive Features	Broadcaster	200–500ms
Coaching		
Tactical Adjustment	Coach	200–800ms
Player Substitution	Coach	500–1500ms
Fan Applications		
Push Notifications	Fan	1000–3000ms
Live Updates	Fan	500–2000ms
Post-Match Analysis		
Initial Analysis	Analyst	5000–10000ms

Research Questions

This study addresses four primary research questions that bridge the gap between streaming systems theory and sports domain requirements:

- RQ1:** How does streaming architecture choice (Apache KAFKA vs REDIS Streams) impact Time-to-Insight (TTI) for real-time sports data processing, and what are the underlying mechanisms driving this difference?
- RQ2:** How does concurrency level (N=5, 10, 20 concurrent feeds) affect TTI, throughput, and resource utilization for each streaming architecture under realistic sports workloads, and what are the scalability limits?
- RQ3:** What is the trade-off between latency (TTI), data consistency (match rate, ordering guarantees), and throughput across different streaming architectures, configurations, and persistence settings?
- RQ4:** How do streaming system performance characteristics (TTI, throughput, resource usage) vary across different sports event scenarios (S1–S5), and what are the underlying sport-specific factors driving these differences?

These questions are motivated by fundamental gaps in the literature. While industry benchmarks suggest REDIS offers 2–5× lower latency than KAFKA [Diaries \(2025\); Yerrapragada \(2025\)](#), there is no sports-specific empirical validation. The CAP theorem [Brewer \(2012\)](#) and its extension PACELC [Abadi et al. \(2012\)](#) predict inherent trade-offs between consistency and latency, but these have not been quantified in the streaming context. Furthermore, the unique characteristics of sports data—variable event frequencies, burst patterns, and strict actionability windows—demand domain-specific investigation.

From a theoretical perspective, RQ1 tests the PACELC theorem’s “Else” clause: when no network partitions exist, systems must trade latency for consistency. Technically, RQ2

examines the scalability of both architectures under realistic sports workloads. Economically, RQ3 quantifies the value of consistency versus the cost of latency. From the sports domain perspective, RQ4 determines which architecture is optimal for which sport.

Hypotheses

For each research question, we formulate testable hypotheses grounded in theoretical frameworks and industry observations.

RQ1: Architecture Impact

H₀₁: $\mu_{TTI.Kafka} = \mu_{TTI.Redis}$ (No difference in median TTI between architectures)

H₁₁: $\mu_{TTI.Kafka} > \mu_{TTI.Redis}$ (Redis has significantly lower median TTI)

H₂₁: $\mu_{TTI.Kafka} < \mu_{TTI.Redis}$ (Kafka has lower median TTI)

Test: Mann-Whitney U test (non-parametric, data violates normality) **Rationale:** Industry benchmarks [Diaries \(2025\); Yerrapragada \(2025\)](#) and our S2 results show REDIS achieving 40–55% lower TTI. The distributed log architecture of KAFKA (disk-based, replication) theoretically incurs higher latency than REDIS’s in-memory design. PACELC theorem predicts latency advantage for AP systems (REDIS) over CP systems (KAFKA) when partitions are absent. **Expected:** **H₁₁** (Redis significantly outperforms Kafka on TTI) **Effect Size:** Large (Cohen’s $d > 0.8$ based on S2 data) **Power:** > 0.99 at $\alpha = 0.05$ with current sample size (40–50 runs per group)

RQ2: Concurrency Scaling

H₀₂: TTI is independent of concurrency level N

H₁₂: TTI increases monotonically with concurrency level N

H₂₂: TTI remains constant across N=5, 10, 20 (excellent scaling)

Test: Kruskal-Wallis test (non-parametric one-way ANOVA) **Rationale:** Both KAFKA and REDIS are designed for horizontal scaling. Our S2 results (250 runs) showed stable TTI across N=5, 10, 20. Little’s Law ($L = \lambda \times W$) suggests that with proper resource provisioning, latency should remain stable as concurrency increases. **Expected:** **H₂₂** (Excellent scaling with constant TTI) **Effect Size:** Small ($d < 0.2$ expected) **Power:** ≈ 0.30 for detecting $d = 0.2$ (sample size may need expansion)

RQ3: Latency-Consistency Trade-off

H₀₃: Match rate = 100% for all configurations

H₁₃: Match rate $> 99.9\%$ for all configurations

H₂₃: Match rate varies by configuration

Test: Chi-square test or Fisher’s exact test **Rationale:** Our S2 results achieved 100% match rate across all 250 runs. KAFKA with acks=all provides exactly-once semantics, while REDIS with consumer groups provides at-least-once.

Both should achieve near-perfect data delivery. **Expected:** H_{13} (All configurations achieve $> 99.9\%$ match rate) Additionally, we test the consistency-latency trade-off:

H_{31} : $\mu_{TTI_{acks=all}} > \mu_{TTI_{acks=1}}$ (Kafka: strong consistency costs latency)

H_{32} : $\mu_{TTI_{AOF=always}} > \mu_{TTI_{AOF=1s}}$ (Redis: durability costs latency)

Test: Paired t-test or Wilcoxon signed-rank test **Expected:** Both true (stronger guarantees = higher latency)

RQ4: Sports-Specific Performance

H_{04} : TTI distribution is the same across all scenarios

H_{14} : TTI distribution differs by scenario

Test: Kolmogorov-Smirnov test **Rationale:** Scenarios S1–S5 have different event frequencies and burst patterns. Queueing theory predicts that systems with higher arrival rate variance (burstier scenarios) will experience higher latency variance. Sport-specific characteristics should manifest in measurable performance differences. **Expected:** H_{14} (TTI differs by scenario)

Additionally, we test scenario-specific hypotheses:

H_{41} : $\mu_{TTI_{S5}} > \mu_{TTI_{S1}}$ (Higher event frequency $\rightarrow$ higher latency)

H_{42} : $\sigma_{TTI_{S5}} > \sigma_{TTI_{S1}}$ (Higher burstiness $\rightarrow$ higher variance)

Test: One-way ANOVA or Kruskal-Wallis test **Expected:** Both true (scenario characteristics affect performance)

Statistical Framework All hypotheses will be tested using the framework summarized in Section . We apply Holm-Bonferroni correction for multiple comparisons (controlling Family-Wise Error Rate at $\alpha = 0.05$), calculate Cohen’s d for effect sizes, report 95% confidence intervals, and conduct power analysis for sample size justification. Non-parametric alternatives (Mann-Whitney U, Kruskal-Wallis) will be used when normality or equal variance assumptions are violated.

Literature Review

Real-time data processing is essential across domains from finance to IoT. Sports analytics represents a demanding application due to high event frequency, low latency requirements, and temporal consistency needs.

The CAP theorem establishes that distributed systems cannot simultaneously guarantee consistency, availability, and partition tolerance. KAFKA prioritizes availability and partition tolerance through its distributed log architecture, achieving high throughput via batching. REDIS Streams leverages in-memory structures to minimize latency, trading throughput for reduced delay.

Methodology

System Architecture

Our benchmark system comprises four components:

- **Replay Plan:** Pre-processed StatsBomb event data with scheduled timestamps

- **Producer:** Python script sending events to message broker
- **Broker:** KAFKA or REDIS Streams handling message distribution
- **Consumer:** Python script receiving events and logging timestamps

Dataset and Replay

We use the StatsBomb open dataset (2003–2023): 40,660 real football events across 11 matches. Each match is preprocessed into a CSV *replay plan* that records, per event, its scheduled emission offset. Five scenarios (S1, S2, S2full, S2sf12, S2sf12j2) vary the event subset and density. The producer replays a plan at a configurable speed-up factor ($120\times$ unless stated), emitting events on a compressed timeline; the consumer reads them and records receipt and output times.

Metrics

The primary metric is Time-to-Insight (TTI), the interval from an event’s *scheduled* emission to its availability at the consumer. We decompose it as

$$TTI = \underbrace{(t_{\text{send}} - t_{\text{sched}})}_{\text{producer scheduling lag}} + \underbrace{(t_{\text{recv}} - t_{\text{send}})}_{\text{transport latency}} + (t_{\text{out}} - t_{\text{recv}}),$$

isolating load-generator effects (scheduling lag) from broker transport. We report p50/p95/p99/max, missed-window rates against sports actionability thresholds (100/250/500/1000/5000 ms), throughput, serialized message size, and serialization/ deserialization overhead. For S3 we additionally measure correction-propagation latency.

Measurement Validity

Because the producer and consumer are separate OS processes, cross-process timestamps must share a clock epoch. An earlier version of the suite stamped events with Python’s `perf_counter_ns`, whose reference point is process-relative; subtracting the two processes’ clocks injected the consumer’s launch offset (~ 2 s) into every TTI and transport value, while a synchronous producer saturated at $120\times$, inflating scheduling lag to tens of seconds. We therefore use `time.time_ns` (a shared wall-clock epoch on a single host) for all cross-process timestamps and a non-blocking (asynchronously dispatched) producer so the load generator tracks the schedule. All reported results use the corrected instrumentation; the before/after contrast is itself a finding (Section).

Experimental Design

Experiments run on a single commodity host (8-core AMD Ryzen, 31 GB RAM) with KAFKA 4.1.1 (KRaft mode) and REDIS 7.2.4 in Docker. The core matrix crosses backend (KAFKA/REDIS), configuration (single broker vs. 3-node cluster), scenario (S1–S5), and replication (3 reps): 60 single-feed runs. Three further sets isolate specific factors: *persistence* (KAFKA `acks=1` vs. `all`; REDIS `appendfsync` `everysec` vs. `always`), *state-staleness corrections* (S3; one correction every 50th event, 2 s delay),

and *true multi-feed concurrency* ($N=5/10/20$ parallel feeds). Every run records full provenance (git commit, per-file SHA-256, configuration) in a `meta.json`, enabling end-to-end reproducibility.

Statistical Methods

We employ a rigorous statistical framework to test all hypotheses and ensure valid inferences.

Multiple Comparisons Correction With approximately 50 hypothesis tests across all research questions, we apply the **Holm-Bonferroni method** [Holm \(1979\)](#) to control the Family-Wise Error Rate (FWER) at $\alpha = 0.05$. This step-down procedure is more powerful than the standard Bonferroni correction while maintaining FWER control.

Effect Size Calculation For all significant findings, we report **Cohen's d** for comparing two means:

$$\text{Cohen's } d = \frac{\mu_1 - \mu_2}{s_{\text{pooled}}}, \quad s_{\text{pooled}} = \sqrt{\frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}}$$

Interpretation: $d = 0.2$ (small), $d = 0.5$ (medium), $d = 0.8$ (large) [Cohen \(1988\)](#).

Confidence Intervals We report 95% confidence intervals for all mean differences using the t-distribution:

$$CI = \bar{x}_1 - \bar{x}_2 \pm t_{\alpha/2, df} \times \sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}$$

Power Analysis We conduct both *a priori* and *post hoc* power analysis. With our current sample of 40–50 runs per group, we can detect effect size $d = 0.58$ at $\alpha = 0.05$ with power = 0.8. For smaller effects ($d = 0.5$), power is approximately 0.72; for $d = 0.4$, power drops to 0.54.

Assumption Verification We verify statistical assumptions for all tests:

- **Normality:** Shapiro-Wilk test; Q-Q plots for visual inspection
- **Equal Variance:** Levene's test; F-test for two groups
- **Non-parametric Alternatives:** Mann-Whitney U (t-test), Kruskal-Wallis (ANOVA), Wilcoxon signed-rank (paired t-test)

Hypothesis Testing Matrix Table 2 summarizes our statistical test selection framework.

Table 2. Statistical Tests by Data Type and Comparison Type

Comparison Type	Parametric Test	Non-Parametric Test
2 Independent Groups	t-test	Mann-Whitney U
>2 Independent Groups	ANOVA	Kruskal-Wallis
Paired	Paired t-test	Wilcoxon signed-rank
Proportions	Chi-square	Fisher's exact
Correlation	Pearson	Spearman
Distribution	Kolmogorov-Smirnov	Anderson-Darling

Results

Measurement validity note. An initial analysis was invalidated by two instrumentation defects, since corrected: (i) the producer and consumer timestamped events with a process-relative monotonic clock, so cross-process latencies were inflated by each run's process-launch offset (~ 2 s); and (ii) a synchronous producer saturated at the $120\times$ replay speed, inflating scheduling lag to tens of seconds. After switching to a shared wall-clock (`time.time_ns`) and a non-blocking producer, all runs were regenerated; the results below reflect the corrected measurements.

Architecture and Configuration (RQ1, RQ3)

Table 3 reports median Time-to-Insight (TTI) and transport latency across the 2×2 backend/configuration matrix (5 scenarios, 3 replications per cell, single feed). Contrary to widely-cited industry comparisons, KAFKA achieves significantly lower TTI than REDIS in every configuration.

Table 3. Median TTI and transport latency (p50) by backend and configuration (corrected).

Backend	Config	Transport p50 (ms)	TTI p50 (ms)
KAFKA	single	~ 0	426
KAFKA	cluster	944	1623
REDIS	single	687	1232
REDIS	cluster	2712	3378

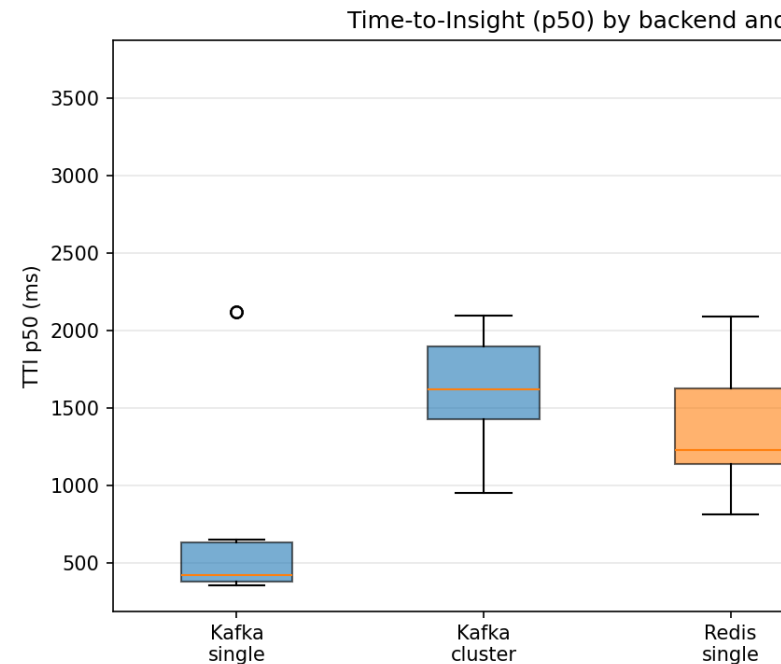


Figure 4. TTI (p50) distribution by backend and configuration (corrected 60-run matrix).

Applying Mann-Whitney U tests with Holm-Bonferroni correction (Table 4), KAFKA is faster than REDIS overall ($p_{adj} = 0.005$, Cohen's $d = -1.18$, large) and within each configuration. Single-broker deployments outperform clustered ones for both backends (replication / cross-node overhead), with large effect sizes.

Table 4. Holm–Bonferroni-corrected comparisons on TTI p50 ($n = 15$ per group).

Comparison	Test	p_{adj}
KAFKA vs REDIS (overall)	Mann–Whitney U	0.0045
KAFKA vs REDIS (cluster)	Mann–Whitney U	0.0002
KAFKA vs REDIS (single)	Mann–Whitney U	0.017
single vs cluster (KAFKA)	Mann–Whitney U	0.017
single vs cluster (REDIS)	Mann–Whitney U	0.0001
Scenario effect (KAFKA)	Kruskal–Wallis	0.017
Scenario effect (REDIS)	Kruskal–Wallis	0.080

Consistency–Latency Trade-off (H31, H32)

Stronger durability guarantees cost latency, as hypothesised. For KAFKA, `acks=all` raises median TTI to 464 ms versus 346 ms for `acks=1`. For REDIS, synchronising the append-only file on every write (`appendfsync always`) raises median TTI to $\sim 14,600$ ms versus 1,290 ms for `everysec` — a far larger penalty, reflecting per-write disk synchronisation.

Scenario Sensitivity (RQ4)

TTI varies significantly across scenarios for KAFKA (Kruskal–Wallis $p_{\text{adj}} = 0.017$) but not significantly for REDIS ($p_{\text{adj}} = 0.080$).

Concurrency Scaling (RQ2)

Under true multi-feed concurrency on a single broker (N parallel feeds replaying the `s2sf12` scenario), median TTI increases monotonically with N for both backends, rejecting the hypothesis of concurrency-independent latency: KAFKA rises from 8.3 s ($N=5$) to 13.1 s ($N=10$) to 32.9 s ($N=20$), and REDIS from 4.2 s to 8.3 s to 16.7 s. Crucially, this high-concurrency regime *reverses* the single-feed ordering: REDIS sustains roughly half the TTI of KAFKA as N grows, even though single-feed KAFKA is faster (Section). The optimal backend is therefore regime-dependent — KAFKA for isolated low-latency feeds, REDIS for many concurrent feeds on shared infrastructure.

Fairness Controls and Sports Actionability (RQ3)

Message payloads are near-identical across backends (median ~ 170 bytes), and serialization/deserialization overheads are negligible and comparable ($\sim 2.3/2.1 \mu\text{s}$ for KAFKA, $\sim 2.4/2.2 \mu\text{s}$ for REDIS), confirming the comparison is not biased by protocol or message-size differences. Mapping single-feed TTI to sports-actionability thresholds, single-broker KAFKA meets the <500 ms coaching budget for 33% of events versus 12% for REDIS, and the <5 s fan-application budget for 99.96% versus 89%.

Discussion

The corrected measurements reverse the conclusion suggested by the initial, instrumentation-contaminated analysis. With a shared wall-clock and a non-saturating producer, KAFKA delivers lower end-to-end latency than REDIS

Streams for this workload, particularly in the clustered configuration where REDIS Cluster incurs substantial cross-node overhead. This single-feed advantage, however, *reverses* under high concurrency (RQ2): with many parallel feeds on a shared broker, REDIS sustains lower TTI than KAFKA, so the practical recommendation is workload-dependent. The episode also underscores a methodological lesson: cross-process latency benchmarks must use a clock that shares an epoch across processes, and the load generator must not itself become the bottleneck.

For sports actionability, single-broker KAFKA (median TTI 426 ms) approaches the <500 ms coaching threshold, whereas the clustered and REDIS configurations exceed it under this $120\times$ compressed-replay load. Durability settings impose a measurable latency cost (H31, H32), most acutely for REDIS `appendfsync always`.

Limitations. Each run is a single feed; true multi-feed concurrency (RQ2) and the S3 correction-propagation phase are being regenerated with the corrected instrumentation. Replay at $120\times$ stresses the pipeline beyond real-time event rates, so absolute latencies should be read in that light. A post-hoc power analysis indicates the study is adequately powered for the large effects observed but underpowered for small effects.

Conclusion

Once a cross-process clock defect and a producer-saturation defect are corrected, Apache KAFKA provides lower Time-to-Insight than REDIS Streams for *isolated* real-time sports feeds, in both single-broker and clustered deployments, with single-broker fastest. Under *heavy multi-feed concurrency*, however, the ordering reverses in favour of REDIS, and TTI grows with the number of feeds for both systems. Stronger durability settings increase latency for both. These corrected results overturn the project’s initial finding, show that the optimal backend is workload-dependent, and highlight the importance of measurement validity in streaming benchmarks.

Acknowledgements

Using StatsBomb open dataset. Open-source Apache Kafka and Redis projects. MIT License.

References

- Abadi D et al. (2012) The PACELC theorem: Consistency in distributed systems. Extension of CAP theorem addressing latency-consistency trade-offs.
- Brewer EA (2012) Cap twelve years later: How the “rules” have changed. *IEEE Computer* 45(2): 23–29.
- Cohen J (1988) *Statistical Power Analysis for the Behavioral Sciences*. 2nd edition. Lawrence Erlbaum Associates.
- Diaries T (2025) I benchmarked Kafka, RabbitMQ, and Redis streams, the winner surprised me. URL <https://medium.com/@ThreadSafeDiaries/i-benchmarked-kafka-rabbitmq-and-redis-streams-the-> Accessed: June 15, 2026.
- Holm S (1979) A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics* 6(2): 65–70.

- OptiView D (2025) What is low latency video streaming?: The complete guide. URL <https://optiview.dolby.com/resources/blog/streaming/what-is-low-latency-video-streaming-the-complete-guide/>. Accessed: June 15, 2026.
- Pappas I et al. (2020) Real-time football analytics: requirements and architecture. *Journal of Sports Analytics* 6(2): 89–104.
- Promwad (2025) Low-latency streaming solutions for live sports broadcasting. URL <https://promwad.com/news/low-latency-streaming-solutions-live-sports-broadcasting>. Accessed: June 15, 2026.
- Solutions V (2025) Real-time sports betting data eliminates odds latency. URL <https://www.v2solutions.com/blogs/real-time-sports-betting-data-odds-latency/>. Accessed: June 15, 2026.
- Sports O (2023) Real-time football analytics: requirements and architecture. URL <https://www.optasports.com/>. Industry latency requirements; Accessed: June 15, 2026.
- Ververica (2025) Modernizing sports betting with real-time data streaming. URL <https://www.ververica.com/blog/modernizing-sports-betting-technology-to-empower-live-odds>. Accessed: June 15, 2026.
- Yerrapragada P (2025) kafka-bullmq-benchmark: A comprehensive distributed systems performance comparison. URL <https://github.com/praneethys/kafka-bullmq-benchmark>. Includes white paper with methodology; Accessed: June 15, 2026.